

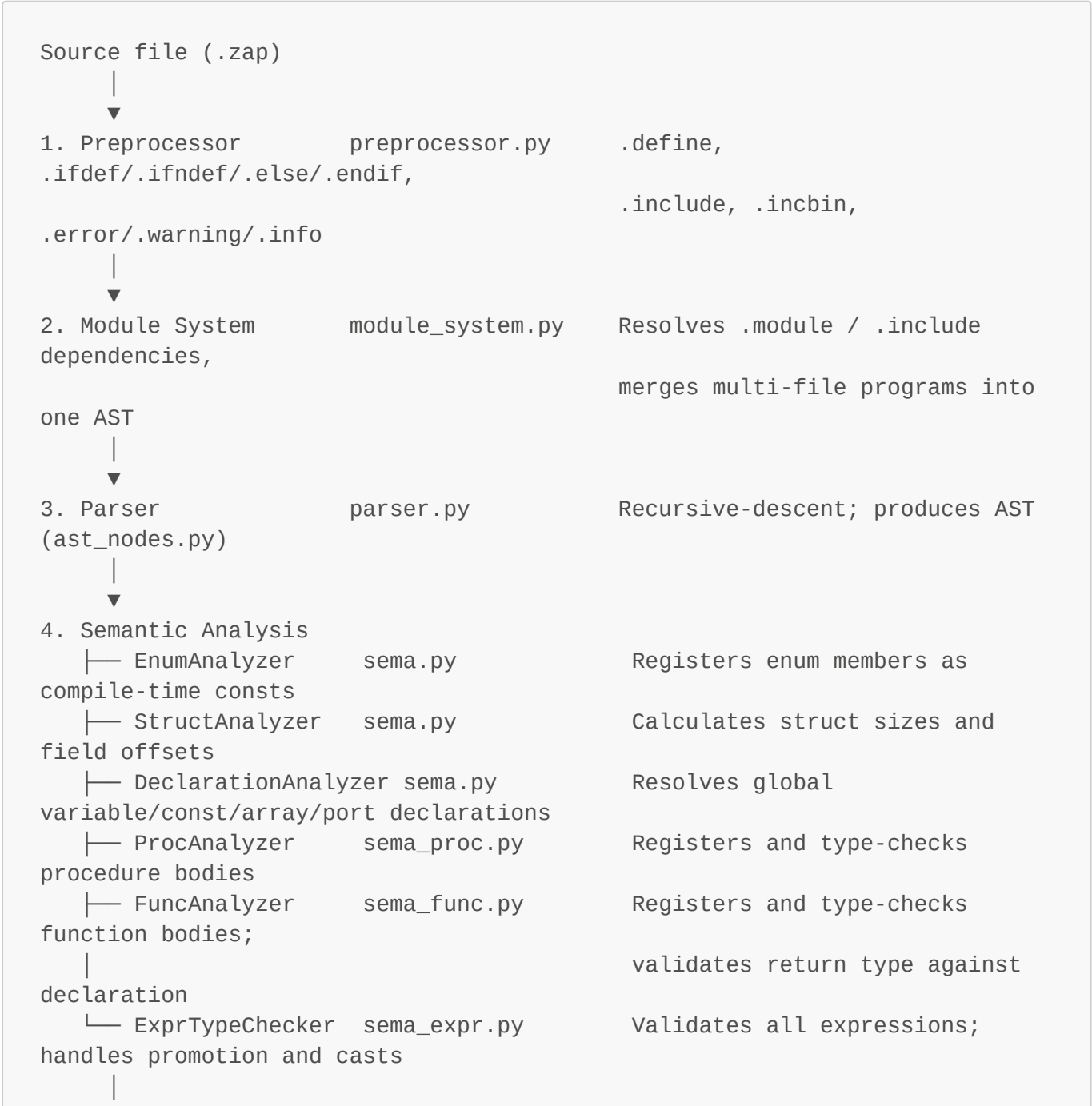
ZAP! Compiler Architecture

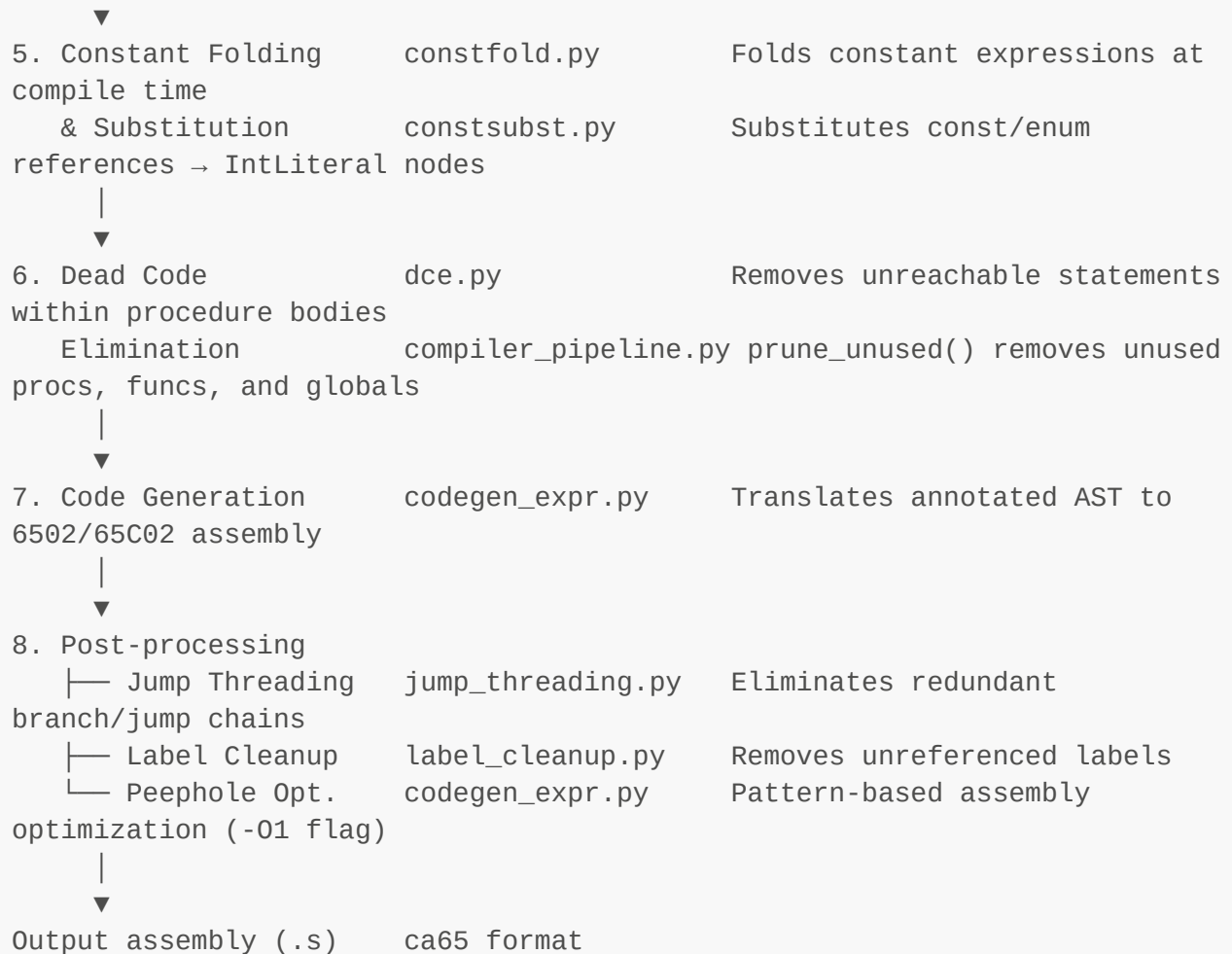
The author of this software stands in solidarity with 🇺🇦 Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in 🇺🇦 Ukraine.

ZAP! is a structured, statically-typed language that compiles to 6502/65C02 assembly for the ca65/ld65 toolchain. The compiler is written in Python and produces `.s` files ready for assembly with ca65.

Compilation Pipeline

The pipeline is implemented in `compiler.py` (entry point) and `compiler_pipeline.py` (orchestration). Each stage is listed in execution order.





Source Files

File	Lines	Role
<code>compiler.py</code>	~235	CLI entry point; parses flags, invokes pipeline
<code>compiler_pipeline.py</code>	~2075	Full pipeline orchestration; <code>compile_program()</code>
<code>preprocessor.py</code>	~125	Macro/include preprocessor
<code>module_system.py</code>	~905	Multi-file module resolution
<code>tokenizer.py</code>	~515	Lexer; produces token stream
<code>token_types.py</code>	~25	Token type constants
<code>parser.py</code>	~1845	Recursive-descent parser → AST
<code>ast_nodes.py</code>	~590	AST dataclass definitions
<code>sema.py</code>	~855	Declaration, struct, and enum analysis
<code>sema_expr.py</code>	~525	Expression type checker
<code>sema_proc.py</code>	~735	Procedure body analysis
<code>sema_func.py</code>	~565	Function body analysis; return type validation

File	Lines	Role
<code>sema_types.py</code>	~25	<code>ExprType</code> / <code>ExprKind</code> definitions
<code>symbols.py</code>	~305	Symbol tables, <code>SemType</code> , <code>StructInfo</code> , <code>ProcSymbol</code>
<code>constfold.py</code>	~130	Constant expression folding
<code>constsubst.py</code>	~65	Const/enum → literal substitution
<code>dce.py</code>	~100	Dead code elimination
<code>jump_threading.py</code>	~90	Jump chain threading
<code>label_cleanup.py</code>	~85	Unused label removal
<code>codegen_expr.py</code>	~13670	Code generation + peephole optimizer
<code>errors.py</code>	~60	<code>SemanticError</code> with file/line/col context
<code>identifier.py</code>	~145	Identifier validation helpers
<code>version.py</code>	1	Version string (<code>0.2.0</code>)

Type System

Types exist in three forms:

AST Level — `TypeNode` (`ast_nodes.py`)

Syntactic type as written in source: base name string + `is_pointer` flag. Examples: `byte`, `word^`, `MyStruct`.

Semantic Level — `SemType` (`symbols.py`)

```
@dataclass(frozen=True)
class SemType:
    base: str          # "BYTE", "WORD", "LONG", or struct name
    is_pointer: bool   # True if ^ (pointer-to)
    is_struct: bool    # True if base is a struct name
    struct_info: Optional[StructInfo] # metadata if is_struct
```

Width rules: pointer → 2, BYTE → 1, WORD → 2, LONG → 4, struct → `struct_info.size`.

Expression Level — `ExprType` (`sema_types.py`)

Wraps `SemType` with an `ExprKind` tag:

- `VALUE` — r-value (literal, temp result, function return)
- `LVALUE` — assignable memory location (variable, array element, struct field)
- `ADDR` — 16-bit address of something (result of `@` operator, or bare array name)

Important: A bare array identifier (used without a subscript) always resolves to `ExprKind.ADDR` with `is_pointer=True` in its `ExprType`, regardless of the element type. Code that needs to distinguish pointer arrays from non-pointer arrays must check the symbol's own `SemType` via `sym.type.is_pointer`, not `expr_type.sem_type.is_pointer`.

Symbol Tables

Structure	Key
<code>SymbolTable</code>	Global and per-procedure variable/const/array symbols
<code>ProcTable</code>	Procedure signatures (name, param count, owner file, exported flag)
<code>FuncTable</code>	Function signatures (name, return type, param count)
<code>StructRegistry</code>	Struct definitions (name → <code>StructInfo</code> with field offsets and sizes)

All lookups are case-insensitive. Local variables shadow globals; `ScopedSymbolTable` chains local + global lookups.

Operator Processing

Handled in `sema_expr.py` by `ExprTypeChecker`:

- **Promotion:** `promote(a, b)` → LONG if either is LONG; WORD if either is WORD/pointer; otherwise BYTE.
- **Pointer arithmetic:** Only `pointer + value` and `pointer - value` are allowed; result is same pointer type.
- **Struct restrictions:** Structs cannot appear in arithmetic or bitwise expressions; only assignment and address-of are valid.
- **Compound assignment** (`+=`, `-=`, etc.): Desugared at parse time in `parser.py:parse_assign()` to `lhs = lhs op rhs`. No sema/codegen changes needed.
- **Array subscript chains:** Intermediate subscripts yield `ADDR`; final subscript yields `LVALUE`.
- **Port access checks:** `#RD`-only variables cannot appear in write contexts; `#WR`-only variables cannot appear in read contexts.

Code Generation

All code generation is in `codegen_expr.py` via the `CodeGen` class (~13,670 lines).

Registers and Zero Page

```

A          8-bit accumulator; primary arithmetic register
X          High byte of 16-bit values (X:A pair)
Y          Loop counter / indirect addressing offset

Zero page (compiler-reserved):
  MATH0    4 bytes – primary math operand / result (LONG-capable)
```

MATH1	4 bytes – secondary math operand
MATH_STACK	32 bytes – spill stack for nested expressions
TMP0-TMP15	2 bytes each – temporary pointers and intermediate values

RPN Expression Evaluation

Expressions are compiled via a Reverse Polish Notation (RPN) evaluator:

1. `ast_to_rpn()` walks the expression AST and produces a flat postfix list of `RPNNode` objects.
2. `rpn_eval_to_code()` processes the list left-to-right, tracking the current value in A/X and spilling to `MATH0/MATH1/MATH_STACK` when needed.
3. For complex sub-expressions the result is stacked; the final value is returned in A (8-bit) or X:A (16-bit/pointer).

This approach reduces code size by ~22% compared to naive recursive code generation.

Fast Paths

Before entering the general RPN path, `gen_expr()` checks for many special cases that produce shorter code:

Pattern	Output
<code>byte_var = byte_var + imm</code>	<code>INC/DEC</code> or <code>LDA; ADC #imm; STA</code> (3 instr)
<code>word_var = word_var + imm</code>	<code>CLC; LDA; ADC #lo; STA; LDA; ADC #hi; STA</code>
<code>var = var << N</code> (byte, N=1)	<code>ASL addr</code> (1 instr)
<code>var = var << N</code> (word)	<code>N × ASL addr; ROL addr+1</code>
<code>var = var & imm</code>	<code>LDA; AND #imm; STA</code> per byte
<code>if byte_var == imm</code>	<code>LDA; CMP #imm; BEQ/BNE</code> (no temp)

Math Runtime Routines

Complex operations are delegated to generated subroutines appended to the output:

Routine	Operation
<code>MUL8</code>	8-bit unsigned multiply
<code>MUL16</code>	16-bit unsigned multiply
<code>MUL32</code>	32-bit unsigned multiply
<code>DIV8</code>	8-bit unsigned divide/modulo
<code>DIV16</code>	16-bit unsigned divide/modulo
<code>DIV32</code>	32-bit divide/modulo
<code>ADD32</code>	32-bit addition

Routine	Operation
<code>SUB32</code>	32-bit subtraction
<code>MOD32</code>	32-bit modulo
<code>LSHIFT32</code> / <code>RSHIFT32</code>	32-bit shifts by ≥ 5
<code>COPY_BYTES</code>	Byte copy; X = count (1–255), TMP0=src, TMP2=dst
<code>COPY_BYTES16</code>	Byte copy; TMP4/TMP4+1 = 16-bit count, for blocks > 255 bytes

LONG-target widening: When the assignment LHS is `long`, all arithmetic and bitwise sub-expressions (even byte/word operands) are promoted to 32-bit and evaluated via the `*32` routines. The compiler tracks natural operand widths (`_nat_*_width`) to avoid reading beyond each variable's actual memory footprint; zero-extension of narrower values to 4 bytes is handled by `emit_move_to_math`. Result is always placed in `MATH0` (4 bytes).

Array Initialization

Arrays with `ListInit` or `StringInit` initializers are stored as ROM data and copied to BSS RAM at program start. Copy strategy:

Array size (bytes)	Strategy
≤ 8	Inline indexed stores (<code>LDA rom_label+n; STA ram_label+n</code>)
9 – 255	<code>COPY_BYTES</code> subroutine (8-bit count in X)
≥ 256	<code>COPY_BYTES16</code> subroutine (16-bit count in TMP4/TMP4+1)

Array Assignment (Runtime Copy)

`dst_arr = src_arr` (where both are array identifiers of the same type and dimensions) is dispatched by `gen_assign()` to `_gen_string_copy()`. This loads TMP0=src address, TMP2=dst address, then:

Total bytes	Runtime call
1 – 255	<code>LDX #count; JSR COPY_BYTES</code>
≥ 256	<code>LDA/STA TMP4/TMP4+1 = count; JSR COPY_BYTES16</code>

Supported element types: BYTE, WORD, LONG, and multi-dimensional arrays of any of the above. Assignment between arrays-of-structs is rejected at compile time with an error.

Struct Copy

Three struct copy paths exist in `gen_assign()`:

Source	Handler
Const struct $\rightarrow$ var	<code>_gen_const_struct_copy()</code> — src is ROM label
Var struct $\rightarrow$ var struct	Inline <code>COPY_BYTES</code> / <code>COPY_BYTES16</code> with symbol addresses

Source	Handler
Function return → var struct	JSR to function; copy from <code>__RETBUF_FUNCNAME</code> (global BSS buffer)

Both the `COPY_BYTES` and `COPY_BYTES16` routines are used for struct copy when the struct size exceeds 255 bytes.

Parameter Passing

Procedures and functions pass arguments via dedicated local variables (`_PROCNAME$PARAM`). Up to two byte-wide parameters may be passed through A and X registers when the optimizer detects no intervening usage (fast-path calling convention). Return values are passed in A (byte) or X:A (word/pointer).

For struct-returning functions, the compiler generates a global BSS buffer `__RETBUF_FUNCNAME` after dead-code pruning. The function writes its result there before `RTS`; the caller copies from the buffer to the destination variable.

Peephole Optimizer (-O1)

The peephole optimizer runs over the generated assembly string before output. It applies a sliding window of pattern-replacement rules:

- **Redundant load elimination:** `STA addr; LDA addr` → removes `LDA` if `addr` not modified between store and load.
- **Dead store elimination:** `STA` immediately overwritten without any intervening read.
- **Jump threading:** `JMP label1; label1: JMP label2` → `JMP label2`.
- **Branch shortening:** `BXX tmp; JMP target` → `BXX_INVERSE target` where branch range allows.
- **INC/DEC pairs:** `INC addr; DEC addr` → eliminated.
- **Redundant LDA #0:** Multiple consecutive `STA` of zero share a single `LDA #$00`.
- **IF condition optimization:** `CMP; BNE tmp; BEQ then` → `CMP; BEQ then`.

Module System

`.module` files export their symbols and procedures for use by including files. The module system:

- Resolves circular dependencies via a visited-set.
- Filters exported symbols based on `#NOEXPORT` / `#EXPORT` attributes.
- Ensures module constructors (`proc X() #KEEP #NOEXPORT`) are always included even if not directly called.
- Propagates struct and enum definitions across module boundaries.

Zero Page Allocation

After semantic analysis, the compiler computes a priority score for each local variable based on loop-nesting depth and access frequency (`zp_priority`). The highest-priority variables are placed in zero page (faster 2-byte addressing) up to the available ZP budget. All other locals use absolute 3-byte addressing.

The ZP budget is determined by the first usable ZP address:

- **-cfg <path>** (recommended): reads the ld65 linker config and automatically extracts the ZP start address from the **MEMORY/SEGMENTS** blocks. This keeps the compiler and linker in sync.
- **-ZPSTART <addr>**: manually sets the first usable ZP address. If both **-cfg** and **-ZPSTART** are given, **-ZPSTART** takes precedence.
- **Default** (neither given): all 256 bytes of ZP are available (**zp_start = 0**).

The available ZP space is always **256 - zp_start** bytes. The compiler detects ZP overflow at compile time and reports a clear error if allocations exceed the budget.

Locals that are live-range-disjoint are also assigned to shared slots (**__LVSL0T_N**) to reduce total memory usage, with structs and arrays going to BSS slots (**__BSSSL0T_N**) and scalars/pointers going to zero page slots.

Enum and Const Architecture

Enums are parsed as **EnumDecl** nodes. **EnumAnalyzer** registers each member as a compile-time **const** symbol. Before code generation, **constsubst.py** replaces all **Identifier** and **FieldAccess** references to consts and enum members with **IntLiteral** nodes. This ensures all constant-valued sub-expressions are visible to **constfold.py** for compile-time evaluation, and that code generation never needs to load const values from memory.

Error Reporting

All errors are reported as **SemanticError** (defined in **errors.py**) with:

- Filename, line, and column (mapped back through preprocessor includes)
- Original source text snippet
- Format: **filename:line:col: error: message**

Supported Targets

Flag	CPU	Notes
<i>(default)</i>	65C02	Enables STZ , PHX/PLX/PHY/PLY , BRA , indirect-indexed improvements
-6502	NMOS 6502	Disables 65C02-only instructions; uses LDA #0 ; STA addr instead of STZ
-01	either	Enables peephole optimizer

Segment Name Options

The three output segment names are configurable via CLI flags. All default to the ca65 standard names.

Flag	Default	Segment
-SEGZ <name>	ZEROPAGE	Zero-page variables (.zp: storage, TMP slots, MATH_STACK)

Flag	Default	Segment
<code>-SEGB <name></code>	<code>BSS</code>	Uninitialized data (scalars that overflow ZP, arrays, structs)
<code>-SEGC <name></code>	<code>CODE</code>	Executable code and runtime helpers
<code>-ZPSTART <addr></code>	<code>0</code>	First usable ZP address; budget = <code>256 - <addr></code> bytes
<code>-cfg <path></code>	<code>(none)</code>	ld65 linker config file; auto-extracts ZP start address

These flags affect every `.segment "..."` directive emitted by the code generator (`codegen_expr.py`) and the segment-detection logic in the pipeline (`compiler_pipeline.py`). The `-cfg` flag is parsed by `cfgparser.py`.